

Universidad Autónoma de Sinaloa
Facultad de Informática
Culiacán



Gestión de Seguridad y Desempeño de Bases de Datos

Alumno: Zamora Serrano Irvin Donovan

Profesor: Fidel Bojórquez Solis

Grado: 4 **Grupo:** 3

Práctica 1: Protección y Privacidad de Datos

Fecha de entrega: 23 de octubre de 2026

Actividad 4 - Anonimizacion

Objetivo

Reducir el riesgo de reidentificación de los clientes generalizando los atributos que podrían identificarlos (edad y código postal), evaluando qué tan únicos son los grupos resultantes y suprimiendo los grupos demasiado pequeños. Se toma como umbral mínimo **k = 5**: cada grupo debe contener al menos 5 personas.

Paso 1. Generalización de la edad

La edad exacta se sustituye por rangos (`18-30`, `31-45` y `46+`), y se cuenta cuántos clientes hay en cada uno.

```
SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
            WHEN edad BETWEEN 31 AND 45 THEN '31-45'
            ELSE '46+' END AS rango_edad,
       COUNT(*) AS total
FROM clientes
GROUP BY 1
ORDER BY 1;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
           WHEN edad BETWEEN 31 AND 45 THEN '31-45'
           ELSE '46+' END AS rango_edad,
       COUNT(*) AS total
FROM clientes
GROUP BY 1
ORDER BY 1;
 rango_edad | total
------------+-------
 31-45      |     4
 46+        |     1
(2 rows)

postgres=#
```

Paso 2. Generalización de la región

El código postal se reduce a sus primeros 2 dígitos, que representan una región más amplia, y se cuenta cuántos clientes hay en cada una.

```
SELECT LEFT(codigo_postal, 2) AS region,
       COUNT(*) AS total
FROM clientes
GROUP BY 1
ORDER BY 1;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# SELECT LEFT(codigo_postal, 2) AS region,
       COUNT(*) AS total
FROM clientes
GROUP BY 1
ORDER BY 1;
 region | total
--------+-------
 80     |     3
 81     |     1
 82     |     1
(3 rows)

postgres=#
```

Paso 3. Combinación de atributos generalizados

Se combinan el rango de edad y la región para observar el tamaño de los grupos cuando se cruzan ambos atributos. Al combinarlos, los grupos se hacen más pequeños y aumenta el riesgo de que una persona quede identificada.

```sql
SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
            WHEN edad BETWEEN 31 AND 45 THEN '31-45'
            ELSE '46+' END AS rango_edad,
       LEFT(codigo_postal, 2) AS region,
       COUNT(*) AS total
FROM clientes
GROUP BY 1, 2
ORDER BY 1, 2;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=#
postgres=# SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
           WHEN edad BETWEEN 31 AND 45 THEN '31-45'
           ELSE '46+' END AS rango_edad,
      LEFT(codigo_postal, 2) AS region,
      COUNT(*) AS total
FROM clientes
GROUP BY 1, 2
ORDER BY 1, 2;
 rango_edad | region | total
------------+--------+-------
 31-45      | 80     |     3
 31-45      | 81     |     1
 46+        | 82     |     1
(3 rows)

postgres=#
```

Paso 4. Identificación de riesgos

Se prueba con rangos de edad más amplios (`18-40` y `41+`) y se ocultan los conteos menores a 5 con la etiqueta `<5`, para identificar los grupos que representan un riesgo de reidentificación.

```
SELECT rango_edad,
       CASE WHEN COUNT(*) >= 5 THEN COUNT(*)::text ELSE '<5' END AS total
FROM (
    SELECT CASE WHEN edad BETWEEN 18 AND 40 THEN '18-40' ELSE '41+' END AS
rango_edad
    FROM clientes
) t
GROUP BY rango_edad
ORDER BY rango_edad;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# SELECT rango_edad,
       CASE WHEN COUNT(*) >= 5 THEN COUNT(*)::text ELSE '<5' END AS total
FROM (
    SELECT CASE WHEN edad BETWEEN 18 AND 40 THEN '18-40' ELSE '41+' END AS rango_edad
    FROM clientes
) t
GROUP BY rango_edad
ORDER BY rango_edad;
 rango_edad | total
------------+-------
 18-40      | <5
 41+        | <5
(2 rows)

postgres=#
```

Paso 5. Supresión de grupos pequeños

Se aplica el umbral k = 5 con `HAVING COUNT(*) >= 5`, de modo que solo se publiquen los grupos con al menos 5 personas.

```
SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
            WHEN edad BETWEEN 31 AND 45 THEN '31-45'
            ELSE '46+' END AS rango_edad,
       LEFT(codigo_postal, 2) AS region,
       COUNT(*) AS total
FROM clientes
GROUP BY 1, 2
HAVING COUNT(*) >= 5;
```

```
PS C:\Users\Buck2\LAB-01-Proteccion-Datos> docker exec -it postgres-lab psql -U postgres
postgres=# SELECT CASE WHEN edad BETWEEN 18 AND 30 THEN '18-30'
            WHEN edad BETWEEN 31 AND 45 THEN '31-45'
            ELSE '46+' END AS rango_edad,
       LEFT(codigo_postal, 2) AS region,
       COUNT(*) AS total
FROM clientes
GROUP BY 1, 2
HAVING COUNT(*) >= 5;
 rango_edad | region | total
------------+--------+-------
(0 rows)

postgres=#
```

La consulta devuelve **0 filas**: con solo 5 registros en la tabla, ninguna combinación de rango de edad y región alcanza el mínimo de 5 personas, por lo que todos los grupos se suprimen.

Conclusión

La generalización de la edad y del código postal reduce la precisión de los datos, pero al combinar ambos atributos los grupos se vuelven pequeños y aumentan el riesgo de reidentificación. Aplicar un umbral mínimo de 5 personas por grupo y suprimir los que no lo cumplen evita publicar información que permita señalar a un individuo. Con un conjunto de datos tan reducido, la supresión elimina todos los grupos, lo que muestra que la anonimización requiere volúmenes de datos suficientes para conservar utilidad analítica.